

Numbers in Digital Systems

Numerical Methods

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory

Department of Electrical and Computer Engineering

Cullen College of Engineering

University of Houston

UNIVERSITY of **HOUSTON** | ECE

Radix

Representing and Converting Base
Binary Numbers and Arithmetic
Bases in Digital Systems



Numerical Bases

- The radix or base is the number of unique digits used to represent a number:

Decimal (base 10)

$$\boxed{} = \boxed{} \times 10^3 + \boxed{} \times 10^2 + \boxed{} \times 10^1 + \boxed{} \times 10^0 = \boxed{}$$

Octal (base 8)

$$\boxed{} = \boxed{} \times 8^3 + \boxed{} \times 8^2 + \boxed{} \times 8^1 + \boxed{} \times 8^0 = \boxed{}$$

Binary (base 2)

$$\boxed{} = \boxed{} \times 2^{11} + \boxed{} \times 2^{10} + \boxed{} \times 2^9 + \boxed{} \times 2^8 + \\ \boxed{} \times 2^7 + \boxed{} \times 2^6 + \boxed{} \times 2^5 + \boxed{} \times 2^4 + \\ \boxed{} \times 2^3 + \boxed{} \times 2^2 + \boxed{} \times 2^1 + \boxed{} \times 2^0 = \boxed{}$$

Hexidecimal (base 16)

10 = *A* 11 = *B* 12 = *C* 13 = *D* 14 = *E* 15 = *F*

$$\boxed{} = \boxed{} \times 16^2 + \boxed{} \times 16^1 + \boxed{} \times 16^0 = \boxed{}$$

Radix Points

- Separates whole numbers from fractions in any base

Decimal (base 10)

$$\boxed{} = \boxed{} \times 10^2 + \boxed{} \times 10^1 + \boxed{} \times 10^0 + \boxed{} \times 10^{-1} = \boxed{}$$

Octal (base 8)

$$\boxed{} = \boxed{} \times 8^2 + \boxed{} \times 8^1 + \boxed{} \times 8^0 + \boxed{} \times 8^{-1} = \boxed{}$$

Binary (base 2)

$$\begin{aligned} \boxed{} &= \boxed{} \times 2^{11} + \boxed{} \times 2^{10} + \boxed{} \times 2^9 + \boxed{} \times 2^8 + \\ &\quad \boxed{} \times 2^7 + \boxed{} \times 2^6 + \boxed{} \times 2^5 + \boxed{} \times 2^4 + \\ &\quad \boxed{} \times 2^{-1} = \boxed{} \end{aligned}$$

Hexidecimal (base 16)

10 = A 11 = B 12 = C 13 = D 14 = E 15 = F

$$\boxed{} = \boxed{} \times 16^1 + \boxed{} \times 16^0 + \boxed{} \times 16^{-1} = \boxed{}$$

Binary Numbers

Registers

Converting Binary Numbers

Binary Arithmetic

Binary

- Numbers are represented as sequences of digits
- Digits define the number of different values that can be represented

min	max		min	max	
0000 ₁₀	9999 ₁₀	→ 10 ⁴ values	0000000 ₁₀	9999999 ₁₀	→ 10 ⁶ values
0000 ₂	1111 ₂	→ 2 ⁴ values	0000000 ₂	1111111 ₂	→ 2 ⁶ values

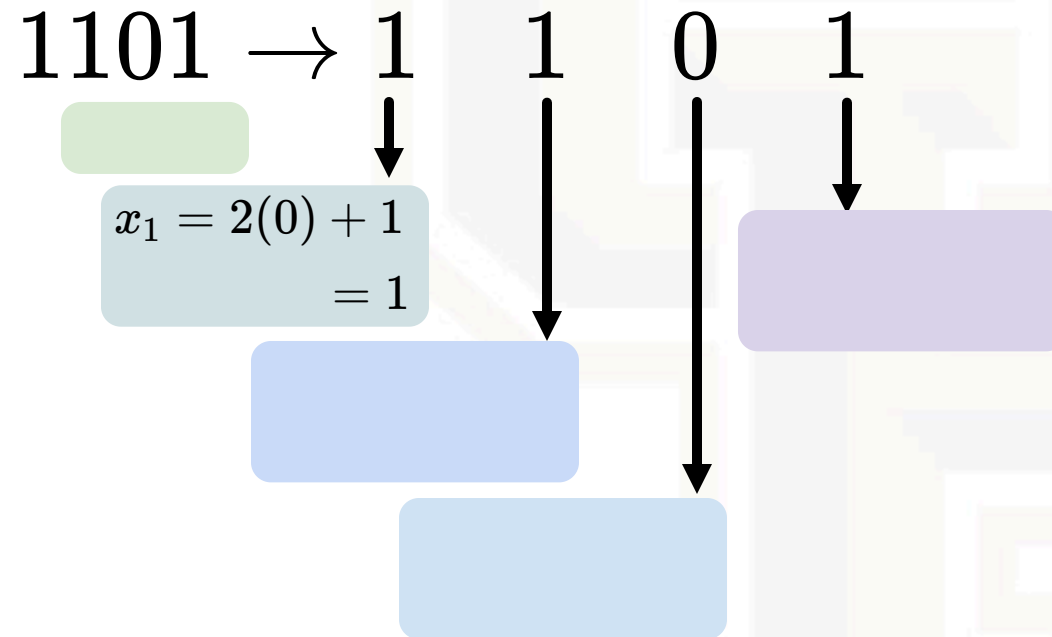
- Modern computers represent numbers using memory cells
- Individual cells can occupy two distinct states: **high** and **low** voltage
- Each cell represents one binary digit: high = 1, low = 0

0100 1101 0110 1010 1111 0011

byte nibble

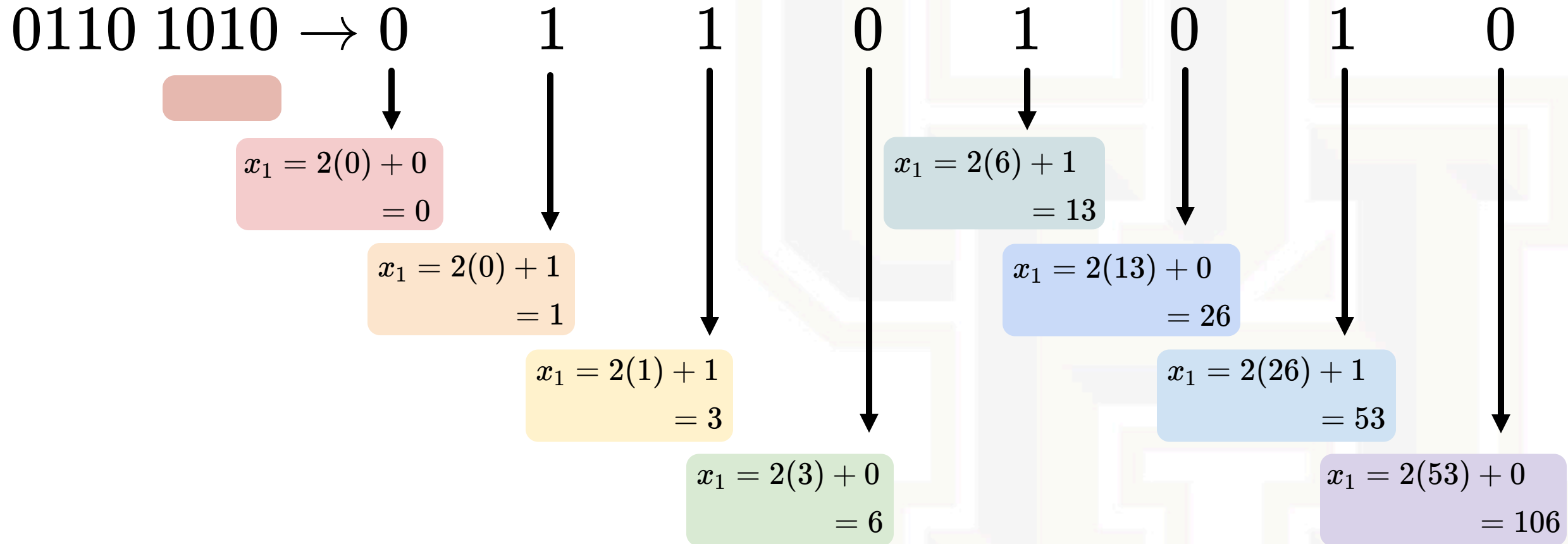
Reading Binary Numbers

1. Initialize a decimal register $x_0 = 0$
2. For each binary digit, double x and add the associated digit as a decimal value

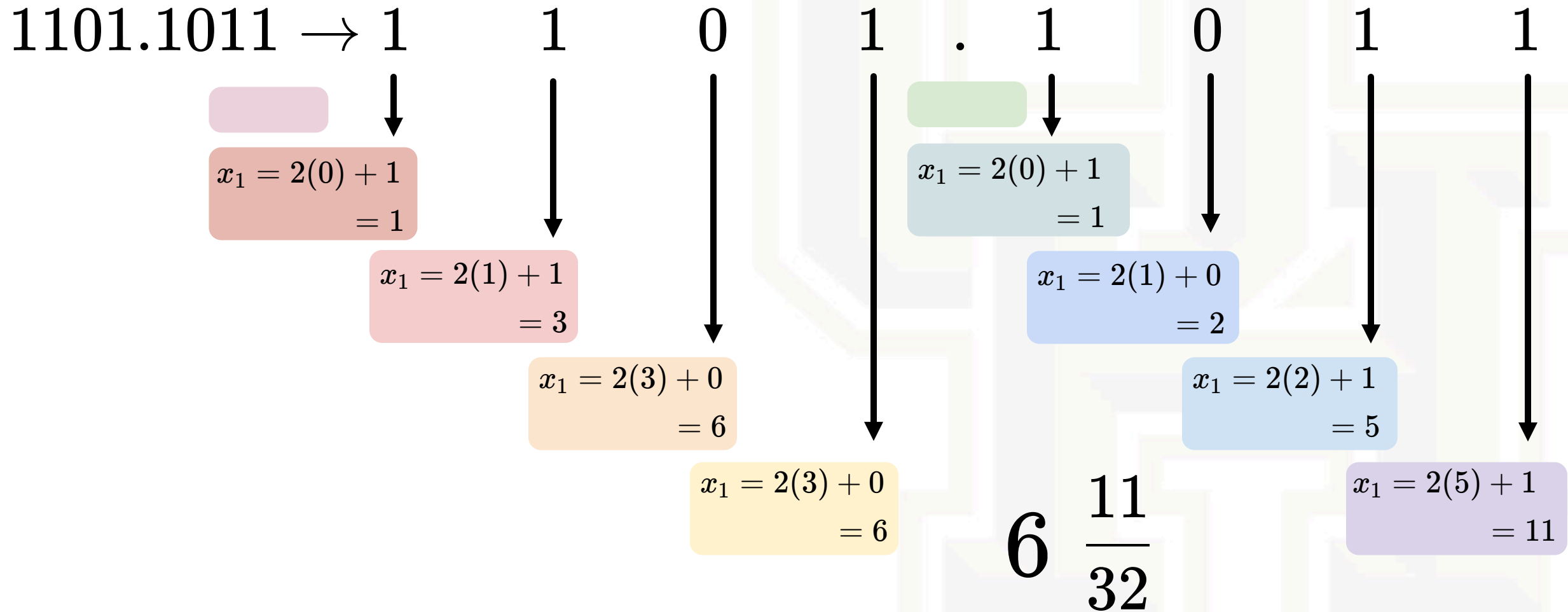


- Used to be known as "double dabble"

Reading Binary Numbers



Fractional Binary Numbers



Binary Arithmetic

- Arithmetic works the same way in any base

$$\begin{array}{r}
 \overset{1}{0} \overset{1}{1} \overset{1}{0} 1 . 0 1 0 \quad 5.250 \\
 + 0 0 1 1 . 1 0 1 \quad + 3.625 \\
 \hline
 1 0 0 0 . 1 1 1 \quad 8.875
 \end{array}$$

$$\begin{array}{r}
 0 1 0 1 . 0 1 0 \quad 5.250 \\
 \times 0 0 1 1 . 1 0 1 \quad \times 3.625 \\
 \hline
 \overset{10}{0} \overset{1}{1} \overset{1}{0} 1 \quad 0 1 0 \\
 \overset{1}{0} 0 0 0 0 \quad 0 0 \\
 \overset{1}{0} 1 0 1 0 1 \quad 0 \\
 \overset{1}{0} 1 0 1 0 1 0 \\
 \hline
 1 0 0 1 1 0 . 0 0 \quad 0 1 0 \quad 19.03125
 \end{array}$$

Representing Integers

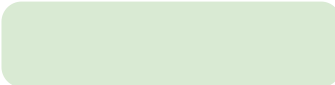
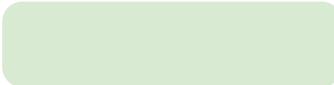
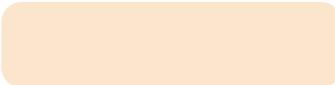
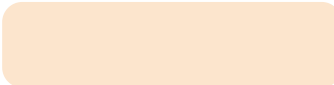
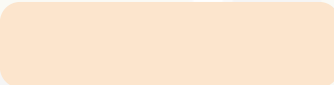
Integers

Signed/Unsigned Integers

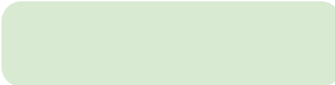
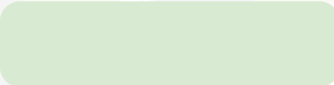
Overflow

Signed and Unsigned Integers

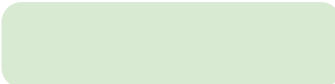
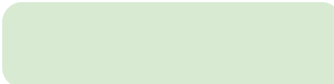
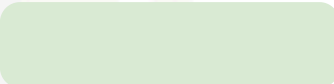
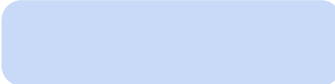
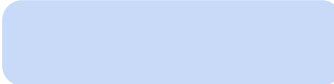
- Represent negative integer values in registers: $-17_{10} = -0001\ 0001_2$
- **Sign and magnitude** - leading bit represents the sign $+$ $\rightarrow 0$ and $- \rightarrow 1$

 = 17_{10}	 = 122_{10}	 = 86_{10}
 = -17_{10}	 = -122_{10}	 = -86_{10}

- **One's complement** - negative values are the bitwise NOT of positive values

 = 17_{10}	 = 122_{10}	 = 86_{10}
 = -17_{10}	 = -122_{10}	 = -86_{10}

- **Two's complement** - negative values are the bitwise NOT + 1

 = 17_{10}	 = 122_{10}	 = 86_{10}
 + 1	 + 1	 + 1
 = -17_{10}	 = -122_{10}	 = -86_{10}

Integer Overflow

- What happens when the result exceeds the available register size?
- Consider a 6 bit addition:
- Overflow is defined for unsigned integers:

1. Assume an n -bit operation
2. Perform the operation
3. Keep the n least significant bits

- Keeping n bits is equivalent to the operation $x \bmod 2^n$: $97 \bmod 2^n = 33$

§6.2.5/9

A computation involving unsigned operands can never overflow, because a result that cannot be represented by the resulting unsigned integer type is reduced modulo the number that is one greater than the largest value that can be represented by the resulting type.

- Overflow is undefined for signed integers

The diagram illustrates a 6-bit addition. On the left, two 6-bit numbers are added: 110101 (53) and 101100 (44). The sum is 100001, where the leading '1' is highlighted in red to indicate overflow. An arrow points from this red '1' to the equation $97 \bmod 2^n = 33$. On the right, the same addition is shown in decimal: 53 + 44 = 97.

110101	53
+ 101100	+ 44
-----	-----
100001	97

What Does This Mean?

- Every register is limited - n bits let you represent 2^n different values
- Signed integers require a signed bit to flag $+/-$ values so the range is halved:

```
unsigned char a;    // [0, 255]
char b;             // [-128, 127]

unsigned int x;     // [0, 2^32]
int x;              // [-2^31, 2^31 - 1]
```

- Most integer overflows are undefined
- Unsigned integers overflow by "wrapping around" to the minimum value:

```
unsigned int a = UINT_MAX + 1; // a = 0
```

- This is the same result as a modulo operation:

```
unsigned int c = a + b;
unsigned int z = (a + b) % pow(2, 32);
// c == z
```

Implementing Floating Point

Floating Point Numbers
Floating Point Arithmetic
Digital Representations

Scientific Notation

- Compressed format using a mantissa m and an exponent n :

$$a = m \times 10^n$$

↑ ↑
mantissa exponent
or **significand**

where $m \in \mathbb{Q}$, $n \in \mathbb{Z}$

- Represent large and small numbers to simplify calculations:

gravitational constant: $G = 6.6743 \times 10^{-11}$

elementary charge: $e = 1.6022 \times 10^{-19}$

speed of light: $c = 2.9979 \times 10^8$

electric permittivity: $\epsilon_0 = 8.8542 \times 10^{-12}$

- Computers and calculators often use "E" to denote the exponent: $6.6743\text{E}-11$

- The mantissa is **normalized**: $1 \leq m < b$ where b is the base

$$G = \boxed{} \times 10^{-14}$$

$$G = \boxed{} \times 10^{-8}$$

$$G = \boxed{} \times 10^{-11}$$

ensures one representation for any value

Floating Point

- Scientific notation can be used in any basis b
- Specify a **precision**: number of digits for m and n
- Mantissa and exponent can be negative or positive

$$a = \pm m \times b^n$$

$$G = \text{[sign]} \cdot \text{[5 digit precision]} \times 10^{\text{[2 digit exponent]}}$$

$$c = \text{[sign]} \cdot \text{[5 digit precision]} \times 10^{\text{[2 digit exponent]}}$$

5 digit precision

2 digit exponent

$$G = \text{[sign]} \cdot \text{[5 digit precision]} \times 2^{\text{[2 digit exponent]}}$$

$$c = \text{[sign]} \cdot \text{[5 digit precision]} \times 2^{\text{[2 digit exponent]}}$$

sign

Floating Point Arithmetic

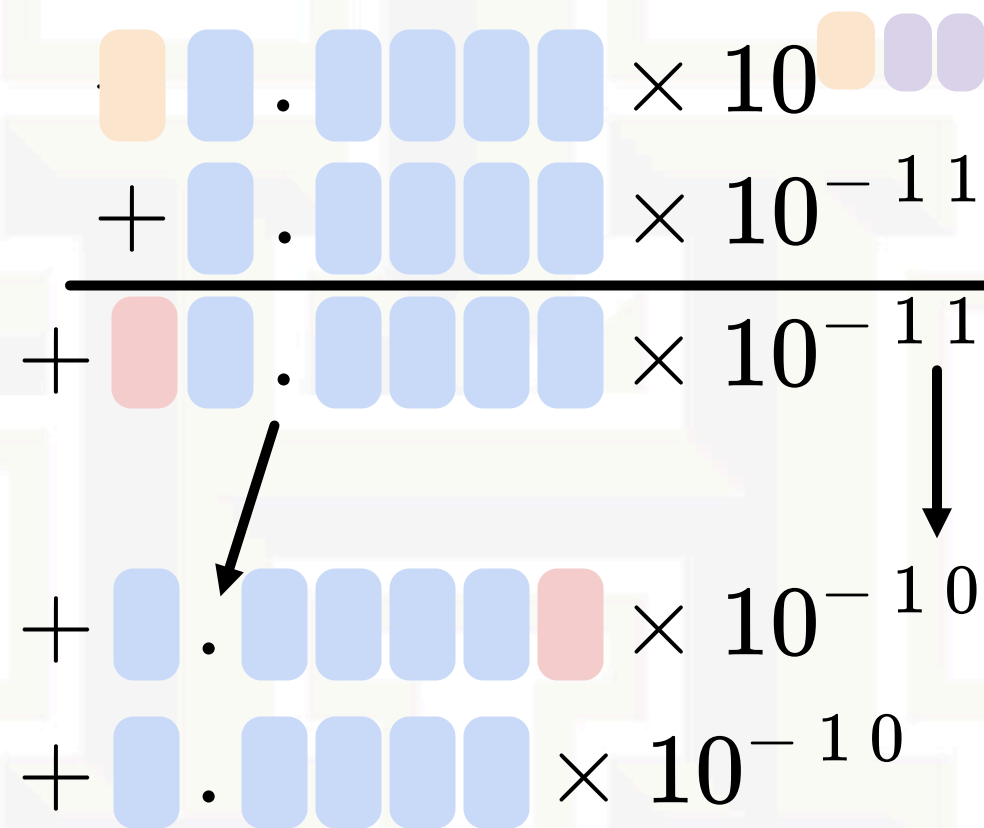
- Fixed precision means that a floating point value $\text{fl}(x)$ may not match the target value x
- A non-representable number x is surrounded by two representable values: $x_- = \lfloor x \rfloor$ and $x_+ = \lceil x \rceil$:

$$1.3144 < 1.31441 < 1.3145$$

- Rounding options:

- round-by-chopping:** $\text{fl}(x) = \begin{cases} \lceil x \rceil & x \leq 0 \\ \lfloor x \rfloor & x > 0 \end{cases}$

- round-to-nearest:** $\text{fl}(x)$ is the closest representable value to x (ties resolve to the closest *even* value)



Implications

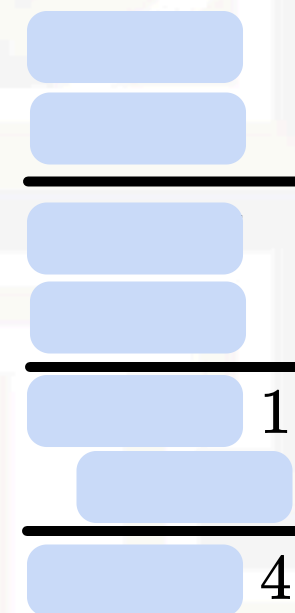
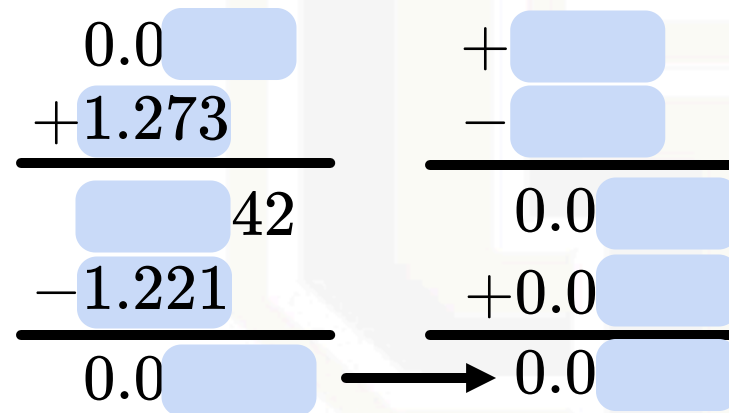
- Operations on floating point numbers are not necessarily associative or distributive:

$$\text{fl}(\text{fl}(x + y) + z) \neq \text{fl}(x + \text{fl}(y + z)) \quad \text{fl}(z \times \text{fl}(x + y)) \neq \text{fl}(\text{fl}(z \times x) + \text{fl}(z \times y))$$

$$(0.03842 + 1.273) - 1.221$$

vs.

$$0.03842 + (1.273 - 1.221)$$



- Cumulative operations can fail:

$$9.993 \times 10^1 + 4.000 \times 10^{-2} = 9.997 \times 10^1$$

$$9.997 \times 10^1 + 4.000 \times 10^{-2} = 1.0001 \times 10^1 \rightarrow 1.000 \times 10^1$$

$$1.000 \times 10^1 + 4.000 \times 10^{-2} = 1.0004 \times 10^1 \rightarrow 1.000 \times 10^1$$

Implementation Quirks

- Digital systems implement floating point using binary values
- Sign-and-magnitude is used for negative/positive values
- Normalization: 1 is always the leading bit, so it doesn't have to be stored (*implied 1*)
- Exponent bias
 - signed exponents are required, but two's complement makes comparisons slower
 - a static bias B is introduced

$$1.m \times 2^{x-B}$$

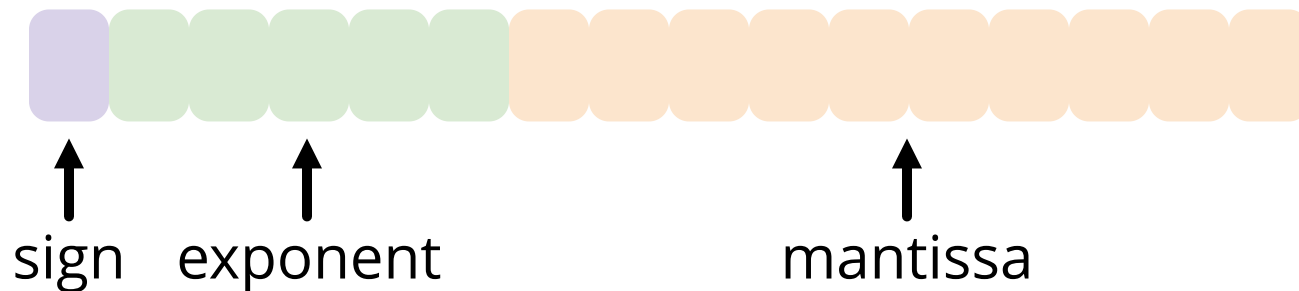
where $m \in \mathbb{Q}$ and $x \in \mathbb{Z}$ are both binary values

Floating Point Standards

- The IEEE 754 standard is the most common for floating point in computing:

standard	C/C++	m bits	x bits	bias
binary16	single	10	5	15
binary32	float	23	8	127
binary64	double	53	11	1023
binary128	N/A	113	15	16383
binary256	N/A	19	237	262143

- Floating point storage within a register:



$$1. \text{ [mantissa] }_2 \times 2^{22-15} = 2^7$$
$$-10110100.010_2 = -180.25$$

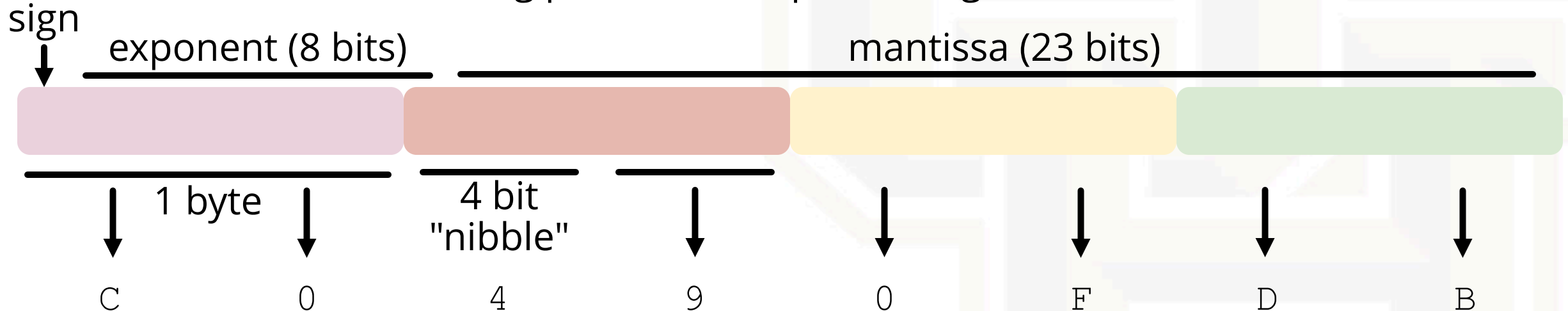
Floating Point Standards (32-bit)

<https://www.h-schmidt.net/FloatConverter/IEEE754.html>

Word Size and Endianness

- The word size is a single unit of data stored, handled by an operation, or transmitted
- The smallest addressable data size is usually a byte (8 bits)

Consider a 32-bit floating point value representing $-\pi$:



- Each 4-bit nibble has $2^4 = 16$ possible values, often represented using hexadecimal
- Bytes in a sequence can be stored in two orders
 - Big-Endian: C0 49 0F DB
 - Little-Endian: DB 0F 49 C0 ← more common

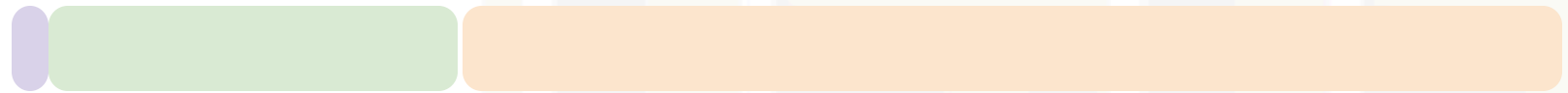
Data Dumps and Memory

<https://hexed.it>

Discussion

- What is stored in this IEEE 754 binary32 register:

A1 1E EF 3B 3B EF 1E A1



$$1. \text{ (orange bar) }_2 \times 2^{127} \text{ (green bar) }_2 \quad 119 - 127 = -8$$

$$+1.86812222003936767578 \times 2^{-8}$$

$$7.2973524220287799835205078125 \times 10^{-3}$$

$$\text{actual } 7.2973525643 \times 10^{-3}$$

$$\epsilon = -1.422712200164794921875 \times 10^{-10}$$